EIGENCOMPRESS

A step-by-step project plan for image compression using Linear Algebra

Project goal
Build a working, interactive web app that compresses grayscale images with low-rank Singular Value Decomposition (SVD), while teaching how eigenvalues, eigenvectors, rank and diagonalisation connect to the mathematics.

Designed for	First-semester Computer Science Engineering student
Experience assumed	Basic Python; learning as you build
Development setup	Android phone + browser-based Replit workspace
Main stack	Python, Streamlit, NumPy, Pillow, Matplotlib; SciPy optional
Recommended duration	10 focused sessions (about 1–2 weeks at a comfortable pace)
Final output	A tested mobile-friendly application, source code, README and demo

Build principle: finish a small, correct end-to-end version first. Add advanced explanations and polish only after upload → compress → compare → download works.

Prepared as an implementation guide

1. Project target and definition of done

The app is an educational demonstrator, not a replacement for production image codecs. Its core function is to approximate a grayscale image matrix using fewer SVD components and let users explore the trade-off between parameter count and reconstruction error.

What the finished app must do

- Accept PNG, JPEG and WebP images, validate them, and display the uploaded image.
- Convert the image to grayscale and cap its dimensions to protect memory and keep processing responsive.
- Compute an SVD-based low-rank approximation and let the user adjust rank k with a slider.
- Display original and reconstructed images, singular values, reconstruction error and theoretical parameter savings.
- Provide a separate small-matrix eigenvalue/eigenvector and diagonalisation explorer.
- Allow users to download the reconstructed image as PNG; JPEG export is an optional extension.
- Explain the math accurately, handle edge cases, and include a README and basic tests.

Definition of done checklist

- A fresh Replit run launches the app without a traceback.
- A sample image works even when the user has not uploaded a file.
- Changing k changes the reconstructed image and updates the metrics.
- Metrics are calculated from the actual arrays, not hard-coded values.
- Invalid files and oversized images produce helpful messages rather than crashing.
- Eigenvalue calculations and diagonalisation checks use a separate small matrix and explain their limits.
- A user can download the reconstructed PNG from a mobile browser.

What is intentionally out of scope for version 1

- No login, database, paid API, cloud AI model, or GPU requirement.
- No claim that SVD output is guaranteed to be smaller than the original encoded file.
- No direct eigen-decomposition of a huge image matrix.
- No promise of production-grade JPEG/PNG compression; this project demonstrates low-rank matrix approximation.

Recommended 10-session roadmap

Session	Main outcome
1	Create Replit project; run a minimal Streamlit page
2	Upload, validate, grayscale and resize an image
3	Implement SVD and reconstruct at a chosen rank
4	Add slider, side-by-side comparison and sample image
5	Add MSE, PSNR and theoretical parameter estimate
6	Add plots for singular values, error and retained energy
7	Build small-matrix eigenvalue and diagonalisation explorer
8	Add downloads, help text, mobile layout and error handling
9	Test edge cases and fix defects
10	Write README, capture demo screenshots and rehearse explanation

2. Understand the mathematics before coding

2.1 Image as a matrix

A grayscale image with m rows and n columns is represented by a matrix A with m × n pixel values. For an 8-bit grayscale image, pixel intensity is typically in the range 0–255. Convert the image to grayscale before building the matrix.

2.2 SVD and low-rank approximation

Singular Value Decomposition represents a real matrix as:

A = U Σ V■

U and V contain orthonormal singular vectors; Σ contains non-negative singular values in descending order. Keeping only the first k components produces:

A■ = U■ Σ■ V■■

- Small k: fewer parameters and usually more visible loss of detail.
- Large k: better reconstruction, but more parameters and more computation.
- For an m × n matrix, a rank-k representation stores about k(m + n + 1) scalar values, versus mn values in the full matrix, ignoring data type and overhead.

2.3 Where eigenvalues and eigenvectors fit

For A■A, which is symmetric and positive semidefinite, eigenvalues are non-negative apart from small floating-point errors. Its eigenvectors give the right singular-vector directions, and the singular values satisfy σ■ = √λ■. In practice, use SVD directly for compression because it is generally more numerically stable than forming A■A.

Do not form A■A for a large image just to demonstrate the relationship: it can be expensive in time and memory. Use a small 2×2 or 3×3 example for the eigen explorer.

2.4 Diagonalisation

A square matrix A is diagonalizable if it has a full set of linearly independent eigenvectors. Then A = PDP■¹, where the columns of P are eigenvectors and D contains their eigenvalues. Not every square matrix is diagonalizable. Diagonalisation is a teaching module in this project; it is not the same operation as SVD-based image compression.

2.5 Metrics

Metric	Formula / interpretation	Caution
MSE	Mean of (original – reconstructed)² over all pixels	Lower is better; 0 means identical arrays
PSNR	10 log■■(255² / MSE) for 8-bit grayscale	If MSE = 0, report infinity or exact match
Parameter reduction	1 – k(m+n+1)/(mn), expressed as %	Theoretical count only; can be negative for large k
Retained squared energy	Σ of retained singular values² / Σ of all singular values²	Not a universal perceptual-quality score

Actual encoded file size depends on PNG/JPEG encoding, metadata, pixel format and preprocessing. Report the original file size and exported file size separately from theoretical SVD parameter count.

3. Step-by-step build: Sessions 1–2

Session 1 — Create and run the starter app

1. **Open Replit** in your Android browser and create a new Python project. Use the current Replit Python template; interface labels may change.
2. **Create a minimal Streamlit app.** Add a heading, a short description and a message confirming the app started.
3. **Add dependencies.** Put streamlit, numpy, pillow and matplotlib in requirements.txt. Add scipy only if you actually use it.
4. **Run the app.** Use Replit's Run control and open the web preview. Resolve dependency or startup errors before adding more features.
5. **Save a checkpoint.** Keep a working version before each major feature. If Git integration is available, commit working milestones.

Starter requirements.txt

```
streamlit\nnumpy\npillow\nmatplotlib
```

Starter app.py

```
import streamlit as st\n\nst.set_page_config(page_title="EigenCompress", layout="wide")\nst.title("EigenC
ompress")\nst.write("Explore low-rank image compression and Linear Algebra.")
```

Success test: the app opens and displays the title without an error. Do not proceed until this works.

Session 2 — Upload and preprocess images

1. Add a file uploader accepting PNG, JPG/JPEG and WebP.
2. If no upload exists, offer a small generated or bundled sample image.
3. Open the image with Pillow inside a try/except block; display a helpful error for invalid content.
4. Convert to grayscale with Pillow's convert('L').
5. Resize only when the longest edge exceeds a chosen cap (start with 400–600 pixels). Preserve aspect ratio.
6. Convert the processed image to a NumPy floating-point array. Keep a copy of the original processed array for metric comparisons.
7. Show the processed dimensions and a note that resizing changes the input independently of SVD compression.

Session 2 acceptance checks

- A small PNG uploads and appears correctly.
- A color photo is displayed as grayscale for the main experiment.
- A very large image is resized without crashing.
- A renamed non-image file shows a useful validation message.
- The app works with the sample image when no file is uploaded.

4. Step-by-step build: Sessions 3–4

Session 3 — Implement the compression engine

Create compression.py

Keep numerical work separate from the Streamlit interface. Implement small functions with clear inputs and outputs:

- **preprocess_image(uploaded_file, max_dimension)** → grayscale image and 2D array.
- **compute_svd(array)** → U, singular values s, and V■.
- **reconstruct_rank_k(U, s, Vt, k)** → reconstructed array.
- **array_to_image(array)** → clipped, rounded 8-bit grayscale image.

Core reconstruction logic

```
U, s, Vt = np.linalg.svd(A, full_matrices=False)\nk = max(1, min(k, len(s)))\nA_k = (U[:, :k] * s[:k]) @ Vt[:k, :]\nA_k = np.clip(A_k, 0, 255)
```

The multiplication U[:, :k] * s[:k] scales each retained column of U by its singular value; multiplying by Vt[:k, :] reconstructs the approximation.

Performance guidance

- Use a maximum image dimension around 400–600 pixels initially; raise it only after measuring performance.
- Use full_matrices=False to avoid unnecessarily large U and V■ arrays.
- Cache SVD results for the current processed image so moving the rank slider does not recompute SVD every time.
- Make sure the cache key changes when the uploaded image or preprocessing size changes.
- Do not store all reconstructed images for all k values; compute only the requested reconstruction.

Session 4 — Build the interactive comparison

1. Add a rank slider from 1 to the number of available singular values. Choose a modest default such as 10, capped to the valid range.
2. Display original and reconstructed images side by side on wide screens; let Streamlit stack columns naturally on narrow screens.
3. Display the chosen k, processed dimensions and theoretical parameter estimate.
4. Add preset rank buttons or a selector for comparing several k values.
5. Ensure that changing the slider refreshes the reconstructed image and associated metrics.
6. Add a reset control that clears the current experiment or restores defaults.

Acceptance checks

- k = 1 gives a very low-rank approximation; higher k values generally improve reconstruction.
- The reconstructed image has exactly the same processed dimensions as A.
- Changing the upload does not show an old image or old metrics.
- The slider never permits k = 0 or k above the valid rank limit.

5. Step-by-step build: Sessions 5–6

Session 5 — Calculate metrics correctly

Create metrics.py

```
mse = np.mean((A.astype(float) - A_k.astype(float)) ** 2)\nif mse == 0:\n    psnr = float('inf')\nelse:\n    psnr = 10 * np.log10((255.0 ** 2) / mse)\noriginal_values = A.shape[0] * A.shape[1]\nretained_values = k * (A.shape[0] + A.shape[1] + 1)\nreduction_pct = 100 * (1 - retained_values / original_values)
```

For small images or high k, the theoretical retained parameter count can exceed the original pixel count. Show the actual calculated percentage, even if negative, and explain that the low-rank representation is then not parameter-efficient.

Display the metrics

- MSE with a brief explanation that lower is better.
- PSNR in decibels; handle exact reconstruction without a divide-by-zero warning.
- Theoretical parameter counts and reduction percentage.
- Actual original upload size and exported PNG/JPEG size, labelled separately.
- A disclaimer that file size depends on encoding and is not equivalent to the SVD parameter estimate.

Session 6 — Add plots

Create visualizations.py and build plots from the computed data rather than illustrative hard-coded values.

Plot	Data and purpose
Singular values	Plot s against component index in descending order.
Cumulative retained energy	Plot cumulative sum(s^2) / sum(s^2) against k.
Reconstruction error	Plot MSE for several valid k values; use a small number of ranks to control runtime.
Parameter count	Compare mn with k(m+n+1) across ranks.
Quality comparison	Display the original and selected reconstruction together.

For error plots, calculate each tested reconstruction from the already computed SVD. Do not run a fresh SVD for every k.

Acceptance checks

- All plots have a title, axis labels and readable legends.
- The singular values appear in descending order.
- Cumulative retained squared energy never decreases and ends near 1 when all singular values are included.
- Metrics and plots update for the currently selected image.
- Plots remain readable on an Android screen.

6. Step-by-step build: Session 7 — Eigenvalues and diagonalisation

This is a separate learning module. It should teach the syllabus directly without slowing down the main image compression path.

Build eigen_analysis.py

1. Provide a few preset 2×2 and 3×3 matrices, including a symmetric matrix and a matrix that is not diagonalizable.
2. Allow manual entry of a small square matrix. Validate that the matrix is square and contains only finite numbers.
3. Use NumPy to calculate eigenvalues and eigenvectors.
4. For each eigenpair (λ, v), compute a residual such as ||Av – λv||. A small residual indicates numerical agreement.
5. Check whether the eigenvector matrix P is invertible and sufficiently well-conditioned before claiming diagonalisation.
6. If valid, compute D and verify that P D P■¹ is close to A within a stated numerical tolerance.
7. If not diagonalizable, explain that the available eigenvectors do not form a full independent basis; do not force a diagonalisation.

Connect this to image compression

For a small processed matrix A, form the symmetric matrix A■A and compute its eigenvalues. Explain that its eigenvectors correspond to right singular-vector directions and that the singular values are square roots of its non-negative eigenvalues, up to floating-point tolerance.

Keep this demonstration small: a 3×3 or similarly small example is sufficient. For real image compression, continue to use SVD directly. Do not construct A■A for a large image because its size and numerical conditioning can make that approach inefficient.

Suggested educational display

- Input matrix A.
- Eigenvalues and eigenvectors.
- Residual for each eigenpair.
- Diagonalisation status and reconstruction check.
- A short plain-language explanation of the result.
- A note that eigenvectors may be scaled and that numerical results can differ slightly because of floating-point arithmetic.

Session 7 acceptance checks

- The selected sample matrix produces the expected number of eigenvalues.
- Eigenpair residuals are small for valid computed eigenpairs.
- A diagonalizable matrix passes the P D P■¹ check within tolerance.
- A non-diagonalizable matrix is not incorrectly reported as diagonalized.
- The module never tries to diagonalize the full image matrix as a requirement for compression.

7. Step-by-step build: Session 8 — Downloads, usability and safeguards

Downloads

1. Convert the reconstructed array to an 8-bit grayscale Pillow image.
2. Write it to an in-memory byte buffer using io.BytesIO.
3. Add a Streamlit download button for PNG.
4. Optionally offer JPEG export with a quality selector, and label the output format clearly.
5. Show the exported file size after encoding. Do not claim that SVD necessarily reduces the actual file size.

Mobile-first usability

- Use concise headings, short explanatory text and Streamlit columns only where they remain readable.
- Keep key controls near the image comparison and avoid extremely wide tables.
- Use expanders for detailed formulas so the main workflow stays uncluttered.
- Add progress feedback for SVD and multi-rank plots.
- Use clear success, warning and error messages.
- Test scrolling, slider operation, image preview and download from the Android browser.

Reliability and safety

- Set reasonable upload-size and image-dimension limits.
- Handle Pillow errors and numerical failures with user-friendly messages.
- Reject empty, malformed or non-finite manually entered matrices.
- Avoid exposing secrets or requesting API keys; this project does not need external services.
- Avoid logging uploaded image contents or retaining user uploads beyond the current session unless there is a clear need.
- Use caching only with keys that identify the actual image and preprocessing configuration.

Recommended file structure

```
app.py\ncompression.py\nmetrics.py\neigen_analysis.py\nvisualizations.py\nrequirements.txt\nREADME.md\ntests/\n  test_compression.py\n  test_metrics.py\n  test_eigen_analysis.py
```

If Replit Agent generates a different structure that is clean and testable, keep it rather than forcing a needless refactor. The key requirement is separation between interface, numerical logic, metrics and tests.

8. Session 9 — Testing and debugging checklist

Run tests on the actual functions and the running application. A page that renders is not proof that the numerical implementation is correct.

Test	Expected result
Small grayscale image	Compression runs; output shape matches input.
Color image	Preprocessing converts it to grayscale.
Invalid file renamed .png	Friendly error; app remains usable.
Very large image	Resize or reject safely before SVD.
k = 1	Valid low-rank output; metrics are finite or intentionally infinite.
k = max rank	Reconstruction is exact up to numerical rounding for the processed matrix.
k beyond valid range	Input is capped or a clear validation message appears.
Constant image	No crash; zero-error PSNR handled correctly if reconstruction is exact.
Eigenpair verification	||Av – λv|| is below a reasonable tolerance.
Diagonalizable matrix	P D P■¹ approximates A within tolerance.
Non-diagonalizable matrix	App reports that diagonalisation is not available.
Switch images	No stale cached image, chart or metrics are shown.
PNG download	Downloaded file opens and has expected dimensions.

Debugging sequence when something fails

1. Read the first traceback error in Replit's console, not just the last line.
2. Identify whether it is a dependency, image-processing, matrix-shape, numerical, or UI-state error.
3. Ask Replit Agent to fix the specific error while preserving already working features.
4. Re-run the smallest test that reproduces the problem.
5. Re-run the full test checklist after changing shared functions.
6. Verify the preview manually on mobile before marking the task complete.

Useful prompt for Replit Agent when debugging

“Inspect the current project and identify the first reproducible error. Explain its root cause briefly, make the smallest correct fix, and run the relevant test. Do not replace real calculations with hard-coded demo values. After fixing it, confirm which test passed and mention any remaining limitation.”

9. Session 10 — README, demo and final presentation

README contents

- Project name and short problem statement.
- Features and supported image formats.
- How to run the app in Replit.
- How to upload an image, select k, interpret metrics and download results.
- Mathematical background: SVD, low-rank approximation, MSE, PSNR, eigenvalues and diagonalisation.
- Testing performed and known limitations.
- Credits and license information for any sample image.

Suggested 3–5 minute demonstration

1. Introduce the problem: storing image detail using fewer mathematical parameters.
2. Upload a sample image and explain how it becomes a grayscale matrix.
3. Set k to a small value and show the reconstruction.
4. Increase k and show the improvement in detail and change in MSE/PSNR.
5. Show the singular-value and error plots.
6. Open the eigenvalue explorer and verify an eigenpair.
7. Demonstrate diagonalisation on a suitable small matrix, then explain why it is not always possible.
8. Download the reconstructed image and summarize the limitations.

What to say about the project's contribution

“EigenCompress is an interactive learning tool that connects first-semester Linear Algebra with image processing. It uses SVD to construct low-rank image approximations and exposes the quality–parameter trade-off. A separate eigenvalue module demonstrates eigenvectors, eigenvalues and diagonalisation, including numerical verification.”

Evidence to collect

- Screenshot of the home page.
- Original and low-rank reconstructed image at two different k values.
- Metrics and singular-value plot.
- Eigenvalue/eigenvector residual display.
- Successful PNG download.
- A brief record of tests and fixes.

10. Replit Agent workflow and useful prompts

Build in small milestones rather than asking the agent to generate and debug everything at once. After each milestone, run the app and verify it before moving on.

Milestone 1 — Starter app

Create a minimal Streamlit app with the planned file structure and requirements. Run it and fix startup errors. Do not implement compression yet.

Milestone 2 — Preprocessing

Implement secure, bounded image upload, grayscale conversion, aspect-ratio-preserving resize and a sample image. Add tests for invalid and oversized images.

Milestone 3 — SVD

Implement the SVD compression engine in a separate module. Add tests for output shape, valid k and reconstruction at maximum rank. Cache the SVD for the current image.

Milestone 4 — Metrics and plots

Implement MSE, PSNR, theoretical parameter estimates, singular-value and retained-energy plots. Calculate all values from actual arrays.

Milestone 5 — Eigen explorer

Add a separate small-matrix eigenvalue/eigenvector and diagonalisation module with residual checks and a non-diagonalizable example.

Milestone 6 — UI and downloads

Complete the mobile-friendly interface, rank slider, image comparison and PNG download. Test changing images and confirm no stale state.

Milestone 7 — Final QA

Run the entire test checklist, fix errors, write the README and report exactly what is implemented and what remains incomplete.

If Replit runs out of context or loses track

Paste a short state summary: what works, which files exist, the first current error, what must remain unchanged, and the next milestone. Ask the agent to inspect the existing project before rewriting anything.

11. Final sign-off checklist

Core functionality

■ App launches in Replit.

■ Upload and sample image work.

■ Grayscale preprocessing and resize work.

■ Rank slider updates reconstruction.

■ PNG download works.

Mathematical correctness

■ SVD reconstruction uses the correct matrix shapes.

■ MSE and PSNR use the processed original and reconstruction.

■ Parameter reduction is labelled theoretical.

■ Eigenpairs are verified numerically.

■ Diagonalisation is only claimed when justified.

Quality and delivery

■ Plots use actual calculated values.

■ Errors are handled gracefully.

■ Mobile preview is usable.

■ README explains setup and limitations.

■ Final demo screenshots are saved.

12. Short glossary

Term	Plain-language meaning
Matrix	A rectangular arrangement of numbers; here, pixel intensities.
Rank	The number of independent rows or columns in a matrix.
Eigenvalue	A scalar describing how an eigenvector direction scales under a square-matrix transformation.
Eigenvector	A nonzero vector whose direction is preserved by a square-matrix transformation.
Diagonalisation	Writing A = PDP■¹ when a matrix has a complete independent eigenvector basis.
SVD	A factorization A = UΣV■ that works for rectangular matrices as well as square matrices.
Low-rank approximation	Approximating a matrix with fewer components, usually trading detail for a more compact representation.
MSE	Average squared pixel difference between original and reconstructed arrays.
PSNR	A logarithmic image-error measure; higher generally means lower pixel error for the same peak range.

Final advice: get the smallest correct version working first, then add one feature at a time. A transparent, tested project with accurate explanations is stronger than a visually impressive interface that reports misleading compression results.